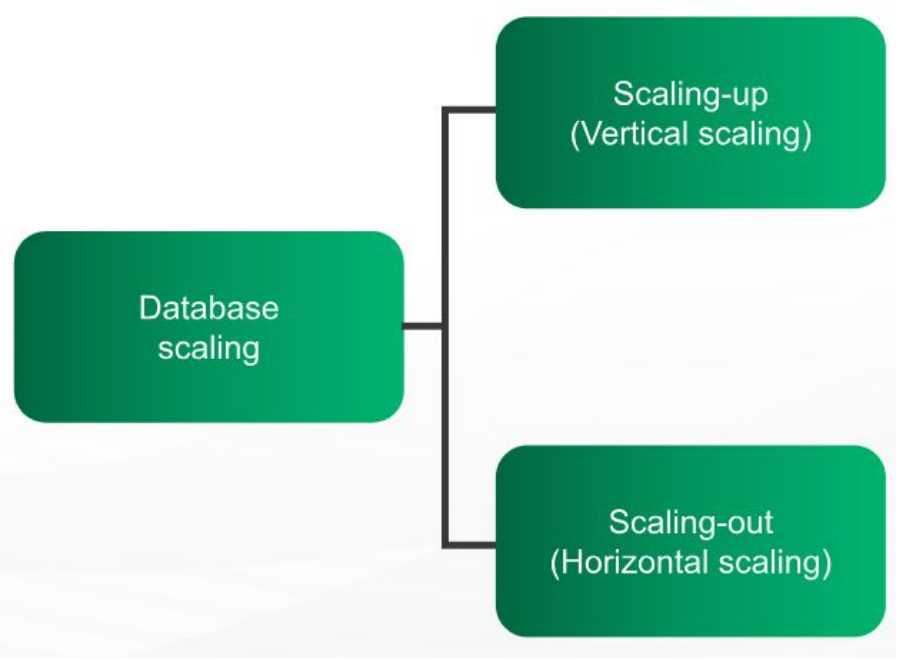
Database Scaling: Replication and Sharding

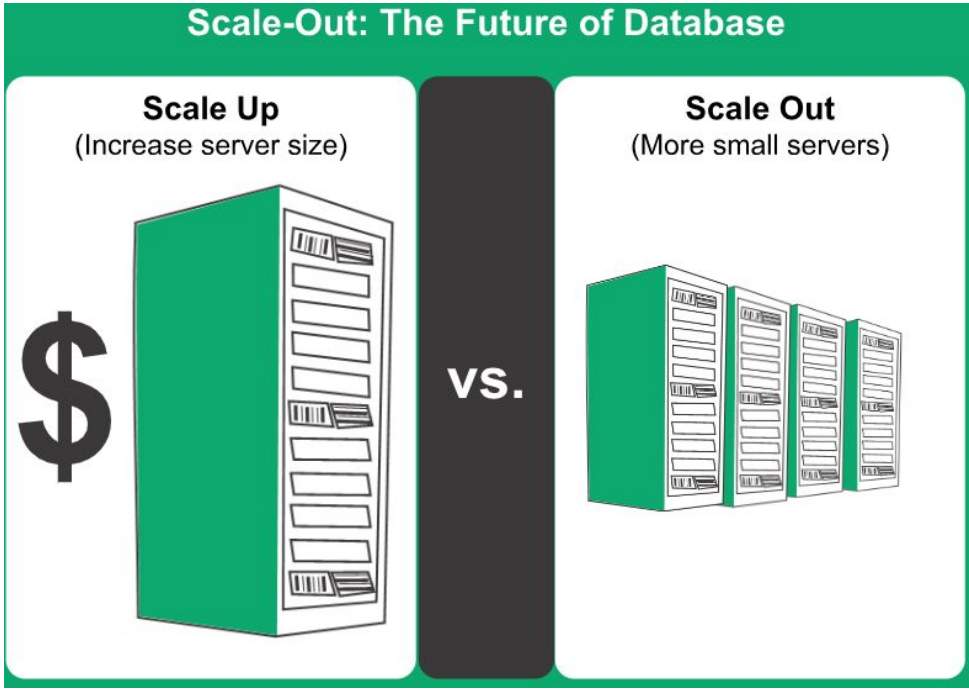
Managing database for availability and performance

- Ability of NoSQL databases to run on a large cluster.
- With increase in data volumes, 'scaling up' becomes difficult. How to ensure database availability?
- 'Scale out' - run the database on a cluster of servers.
- Based on the distribution model, large volumes of data can be handled.

As the amount of data grows by the time, the need for databases to become highly available and efficient is very critical. To make the databases available and perform better, a lot of techniques have evolved over time to improve database performance to handle huge volumes of data.

Database Scaling

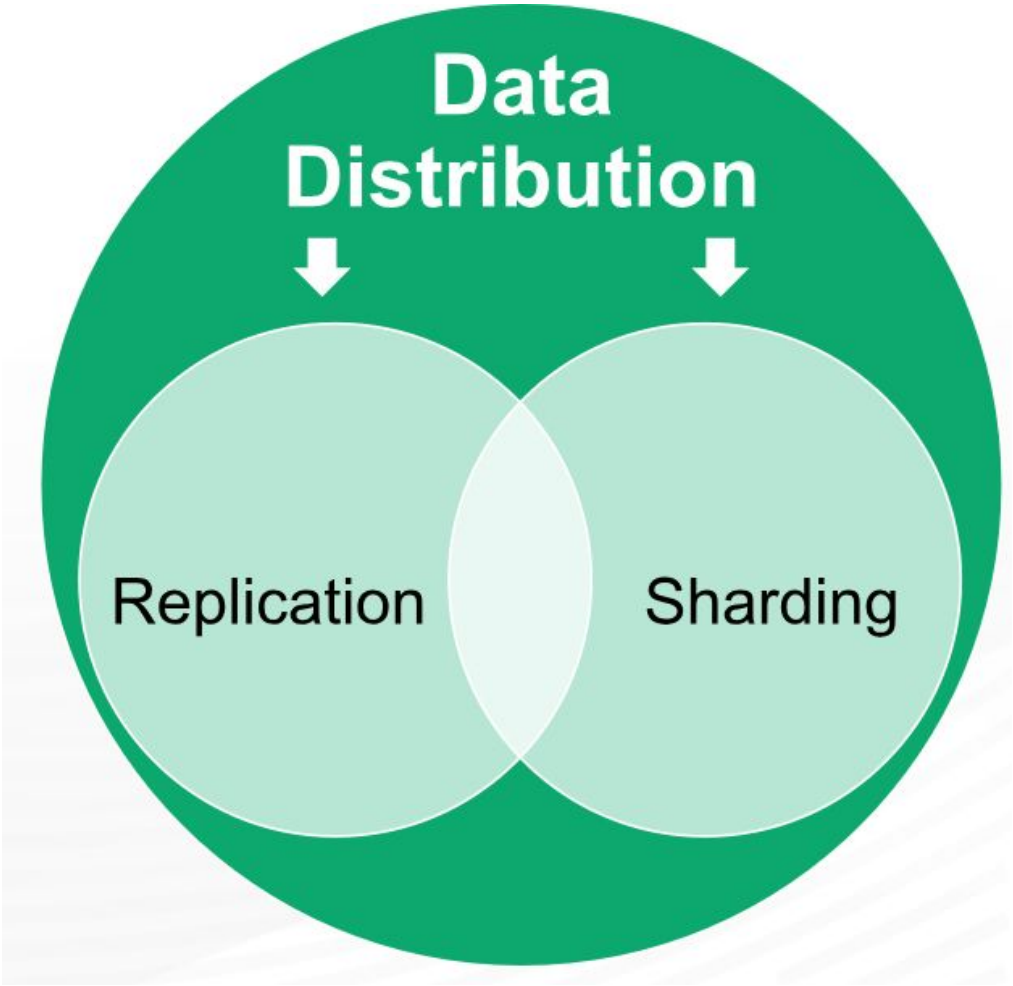




Scaling-up/Vertical scaling: means **upgrading a server**, by means of adding more memory, faster processor, or larger storage devices. Adding more hardware can help increase the amount of traffic a server can support and the amount of data it can process in a reasonable period of time.

Scaling-out/Horizontal scaling: means **adding more servers to the task**. Though, this is complex process, this helps in increasing the capacity to a great extent. In one way, the **additional servers act as backup devices** and play their role when the main server fails. In other ways, **work and network traffic can be shared between all the servers**. When many servers are added to the task, the group of machines assigned to a particular task is called as **cluster.**

Database Distribution Models

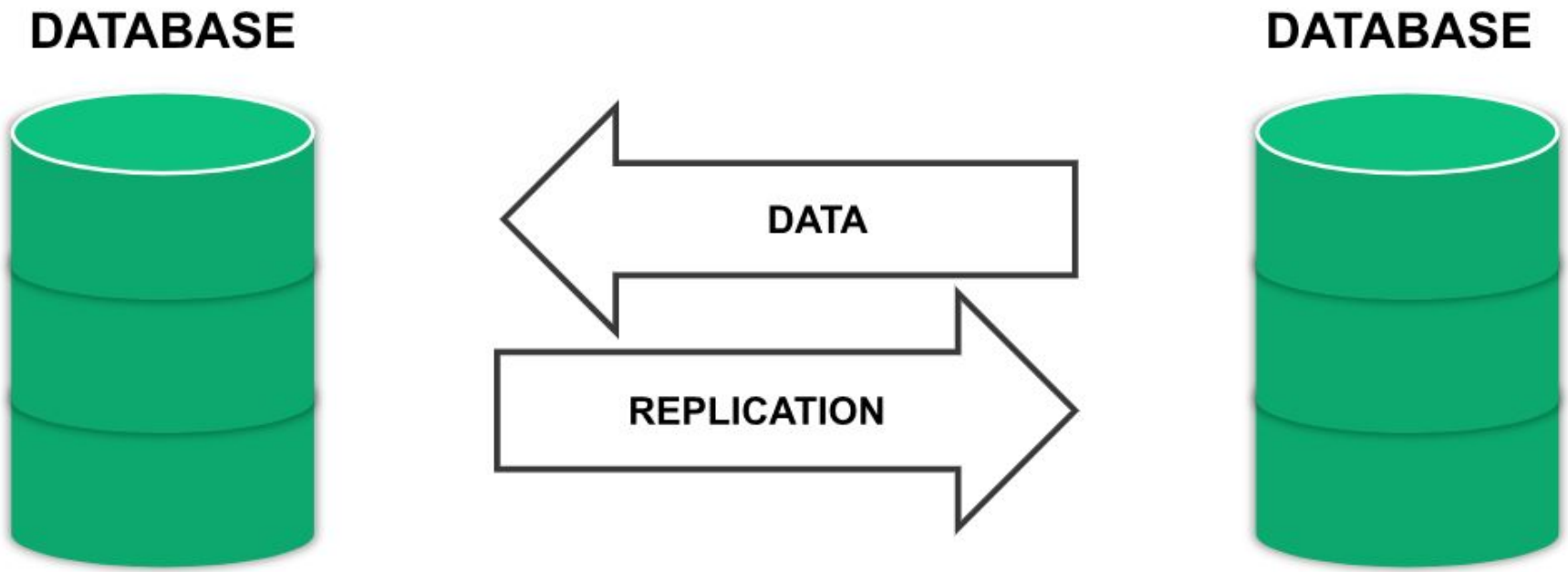


Replication: Same data is copied on multiple servers.

Sharding: Data is put on multiple nodes.

Data Replication

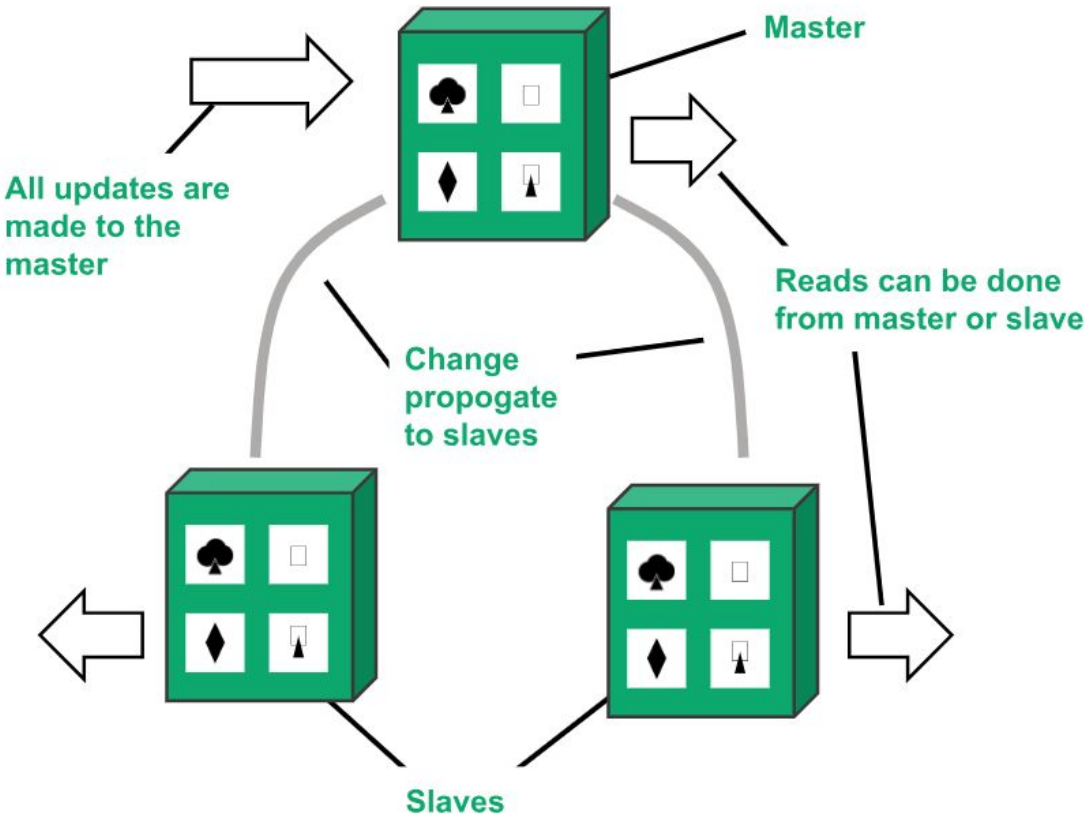
- Process of keeping the data synced.
- Complete copy of database on a different server.
- When the primary server fails, the backup server will come online and take the responsibility.
- Reduced downtime compared to fixing failures manually
- System automatically detects failures and activate backup machines.



Types of database replication

1. Master-slave replication (Passive Replication):

- One master or primary node and others are slaves.
- Master is the primary source of data and designed to process updates and send to slaves.
- Slaves are used for read operation.
- Used for read-intensive datasets.
- Horizontal scaling possible by adding more slaves.



- Masters can be appointed manually and automatically.

Master Slave Replication

- Master-Slave Replication can be **synchronous or asynchronous**. The difference is simply the timing of propagation of changes.

- If the changes are made to the master and slave at the same time, it is synchronous.

- If the changes are queued up and written later, it is asynchronous.

Advantages and Disadvantages of Master-Slave Replication

Advantages

- More Read requests-
  - More slave nodes added.
  - All read requests routed to the slaves.
- Read Resiliency
- Appropriate for read-intensive datasets.

Disadvantages

- Limitations of Master-
  - Master's inability to process updates and pass the updates on to the slaves.
- Inconsistency.
- Not suitable for write-intensive datasets.

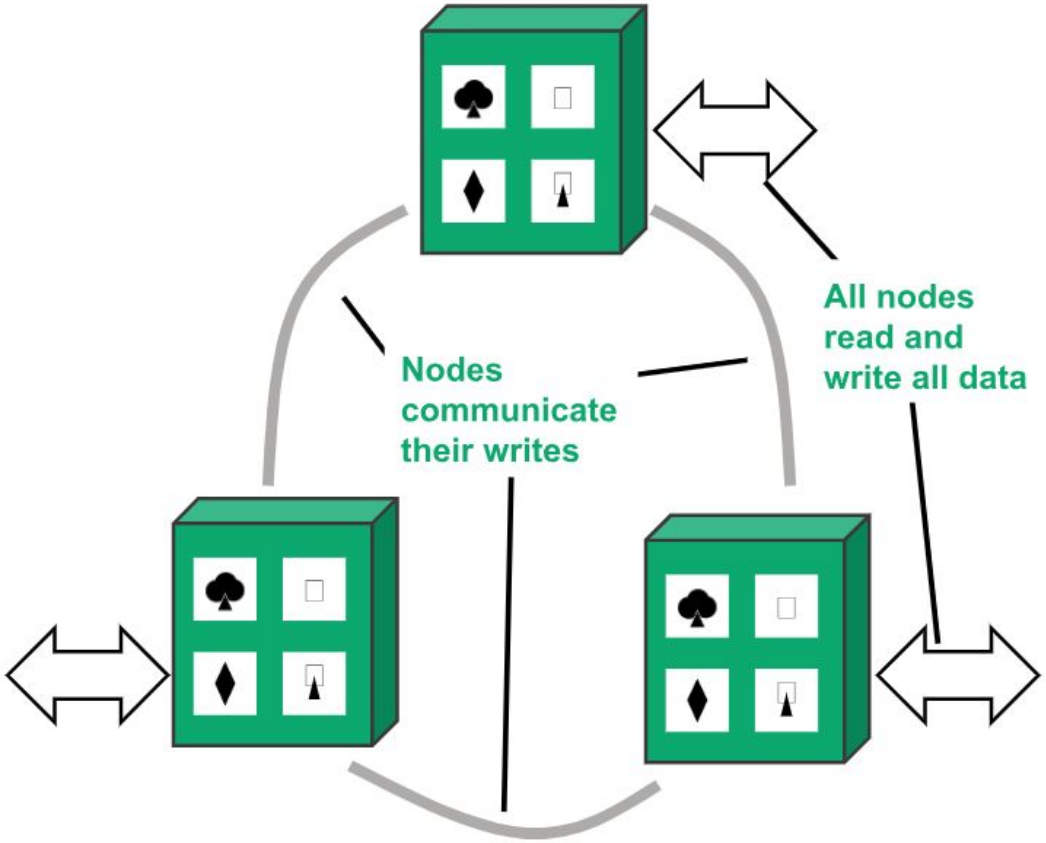
2. Peer-to-Peer Replication (Active Replication):

- Very useful for improving the 'write' scalability that is not offered by master-slave replication.

- There is no master and no single point of failure.

- All the replicas are equal and accept the writes and reads.

- More nodes can be added for performance improvement.



Advantages and Disadvantages of Peer-to-Peer Replication

Advantages

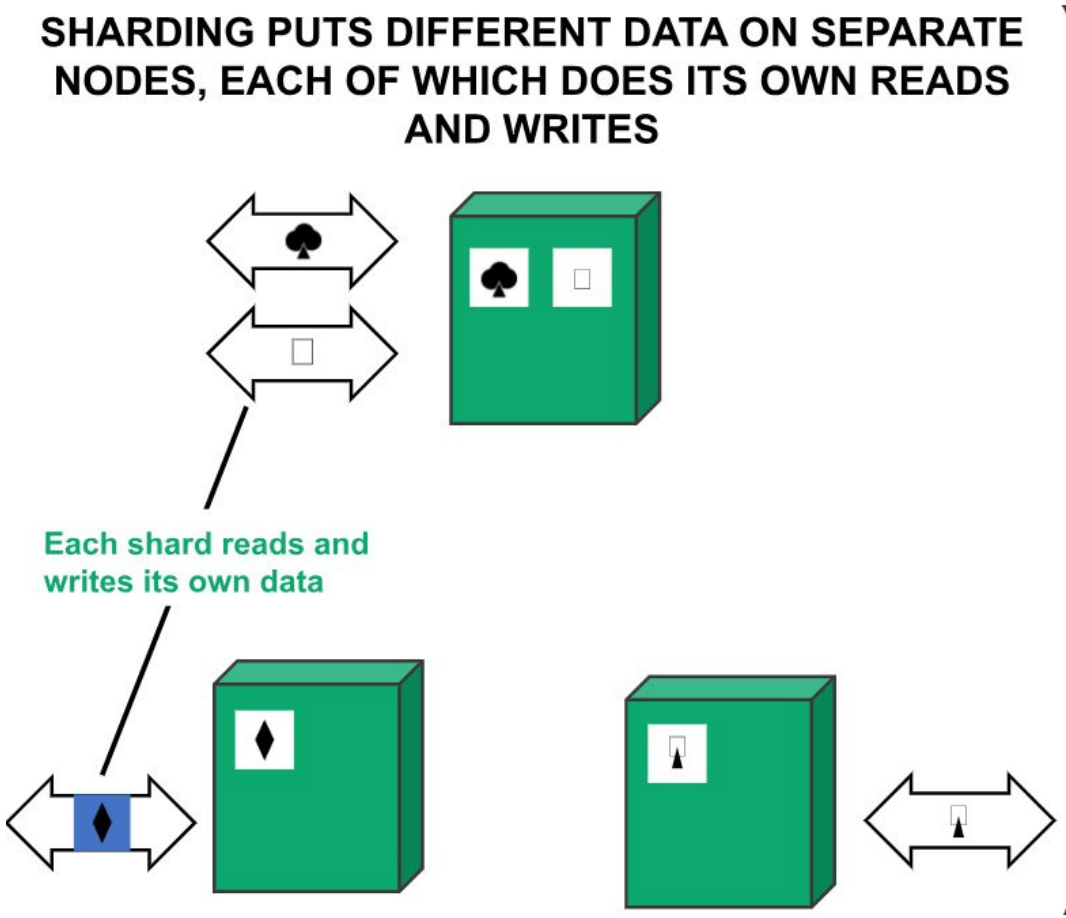
- Node failures can be overridden without any loss of data.
- Nodes can be added easily for performance .

Disadvantages

- Inconsistency.
- Slow propagation of changes to the copies on different nodes.
- Write-write conflict.

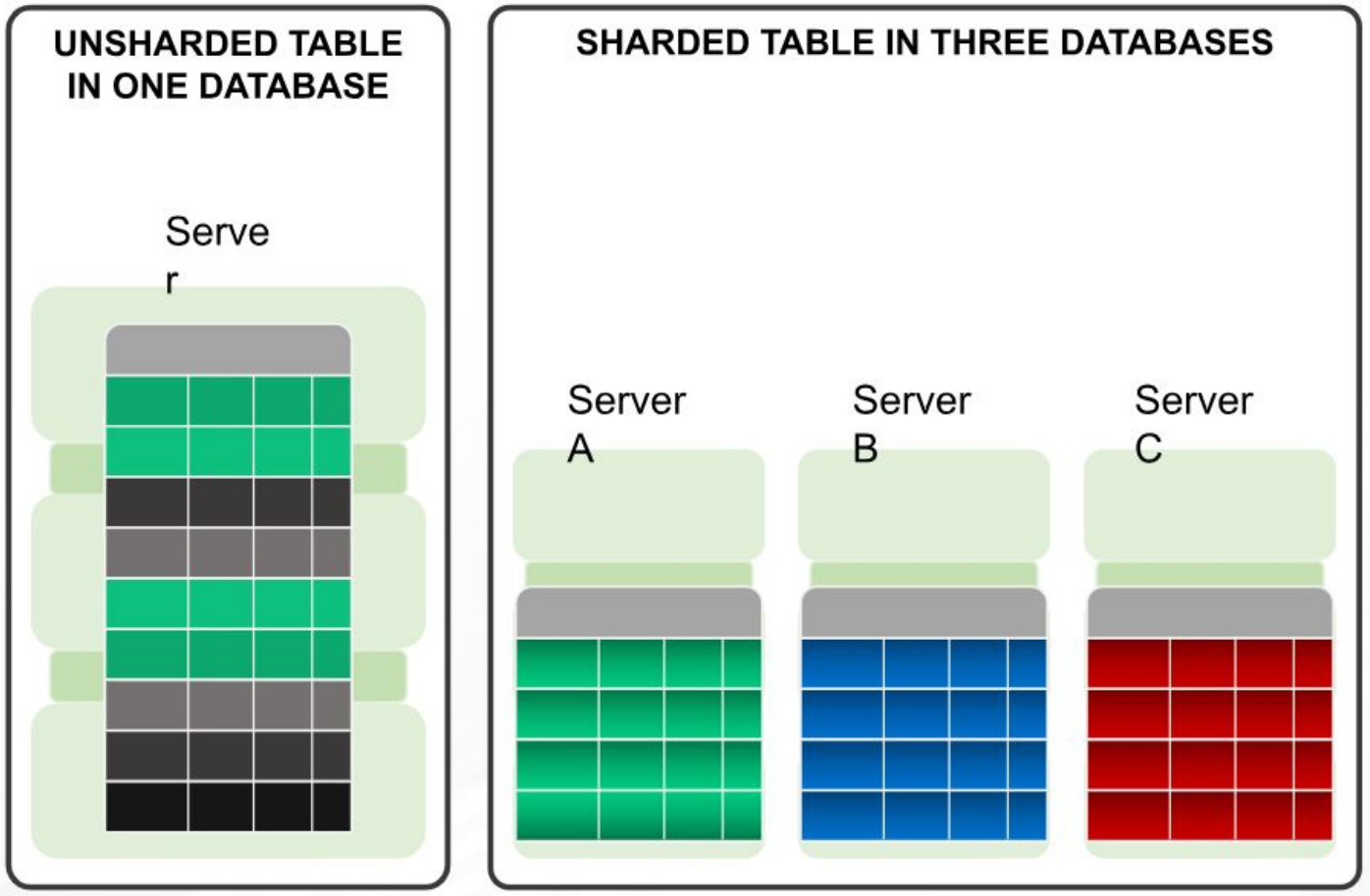
Introduction to Sharding

- Sharding is a method for distributing data across multiple machines.
- Defined as “shared-nothing” partitioning schema for large databases across a number of servers.
- Method of splitting and storing a single logical dataset in multiple databases.
- MongoDB uses sharding to support deployments with very large data sets and high throughput operations.
- Optimization technique to achieve horizontal scaling in DB.



- Database systems with large data sets or high throughput applications can challenge the capacity of a single server.

Why Sharding?

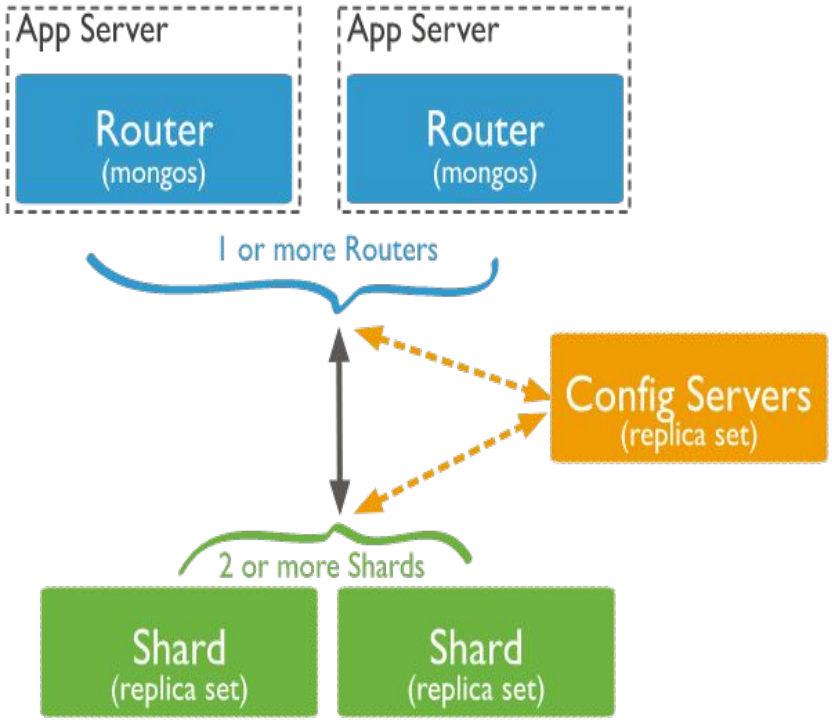


- For improving the throughput and overall performance of database.
- Correct implementation of sharding overrides the setup cost.
- Cost-effective and near-linear scalability for high-volume business transaction applications.
- Write scalability.
- Processing of large datasets.

Sharding components

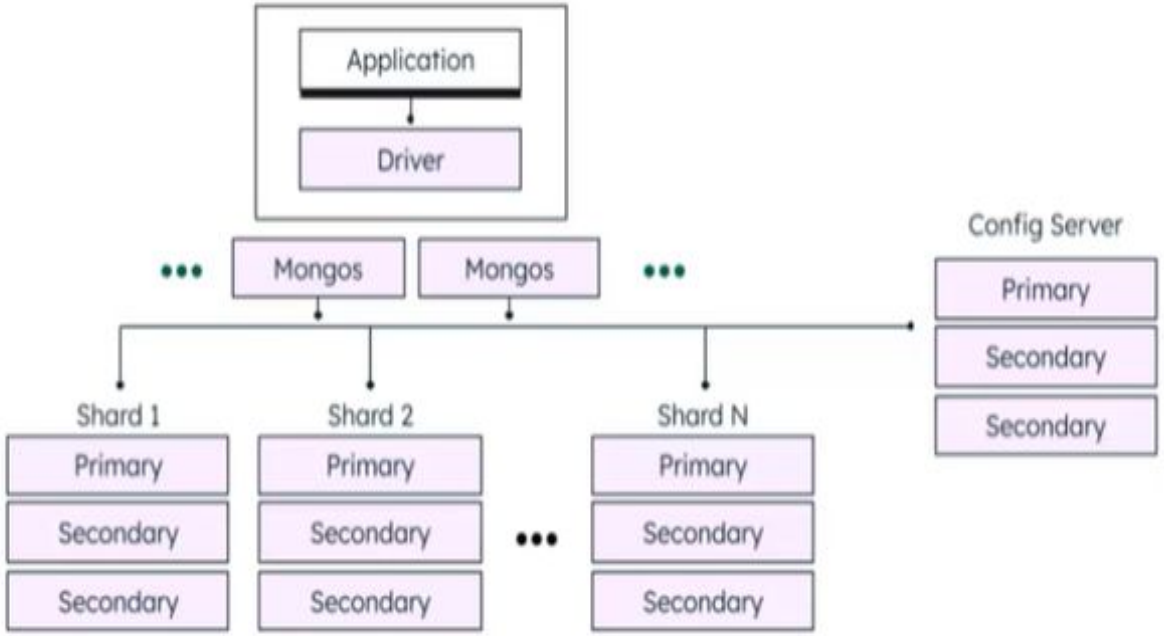
- **Shard:** A shard is a single MongoDB instance that holds a subset of the sharded data. Shards can be deployed as replica sets to increase availability and provide redundancy. The combination of multiple shards creates a complete data set. For example, a 2 TB data set can be broken down into four shards, each containing 500 GB of data from the original data set.

- **Mongos:** Mongos act as the query router providing a stable interface between the application and the sharded cluster. This MongoDB instance is responsible for routing the client requests to the correct shard. Consumes minimal system resources so can be used on existing application servers.



- **Config Servers:** Configuration servers store the metadata and the configuration settings for the whole cluster.

How Sharding Works?



1. The application communicates with the routers (mongos) about the query to be executed.
2. The mongos instance consults the config servers to check which shard contains the required data set to send the query to that shard.
3. Finally, the result of the query will be returned to the application.

How Sharding works?

How will the data be distributed across shards? A data store is divided into horizontal partitions or shards. A shard is a data store in its own way, running on a server, acting as a storage node.

While sharding, it is important to decide which data will be placed on each shard. A shard will contain items that fall in a range that is determined by one or more attributes of data and these are the attributes that form the shard key, also referred to as partition key. The shard key must be static and should not be based on data that changes over time.

Shard or partition Key: MongoDB uses the shard key to distribute the collection's documents across shards.

- The shard key consists of a field or multiple fields in the documents.
- The shard key is either a single indexed field or multiple fields covered by a compound index that determines the distribution of the collection's documents among the cluster's shards.

Logical Shard: is how data is divided based on the shard key. (plan/logic).

Physical Shard: also referred to, a database node, is the actual server or machine where data is stored.

Sharding organizes the data and the sharding logic directs the application to the appropriate shard, when an application stores and retrieves data. This logic could be implemented as part of the data access code in the application or the data storage system implements it if it supports sharding transparently. A Physical location of the data is abstracted in the sharding logic. This ensures control over which shards contain which data. In the event of redistribution of shards later, this enables data to migrate between shards without any need to rework the business logic of the application. The only compromise here is the additional data access overhead that is required in determining the location of each data item as it's retrieved.

For ensuring optimal performance and scalability, it is critical that data is split in a way that is suitable for the queries performed by the application. It is not always the case that the sharding scheme will match the exact requirements of queries. If queries regularly retrieve data using a combination of attribute values, a composite shard key can be defined by linking attributes together.

Steps to shard a collection using a shard key:

- use myDatabase
- sh.enableSharding("myDatabase")
- db.myCollection.createIndex({ userId: 1 }) **//Shard key must be indexed**
- sh.shardCollection("myDatabase.myCollection", { userId: 1 })

{ userId: 1 } → shard key
1 = ascending index

Shard key

- When sharding a MongoDB collection, a shard key gets created as one of the initial steps.

- The “shard key” is used to distribute the MongoDB collection’s documents across all the shards.

- The key consists of a single field or multiple fields in every document.

- **The sharded key is immutable and cannot be changed after sharding** because data is already distributed across shards using the original key and changing it would require re-sharding the entire dataset.
- **A sharded collection only contains a single shard key.**
  - For example,

    sh.shardCollection("ecommerce.orders", { userId: 1 })
  - Now, every document in orders is placed based on userId . MongoDB will **always use this same key**
  - You cannot later say: “Use productId for some documents” and “Use date for others”

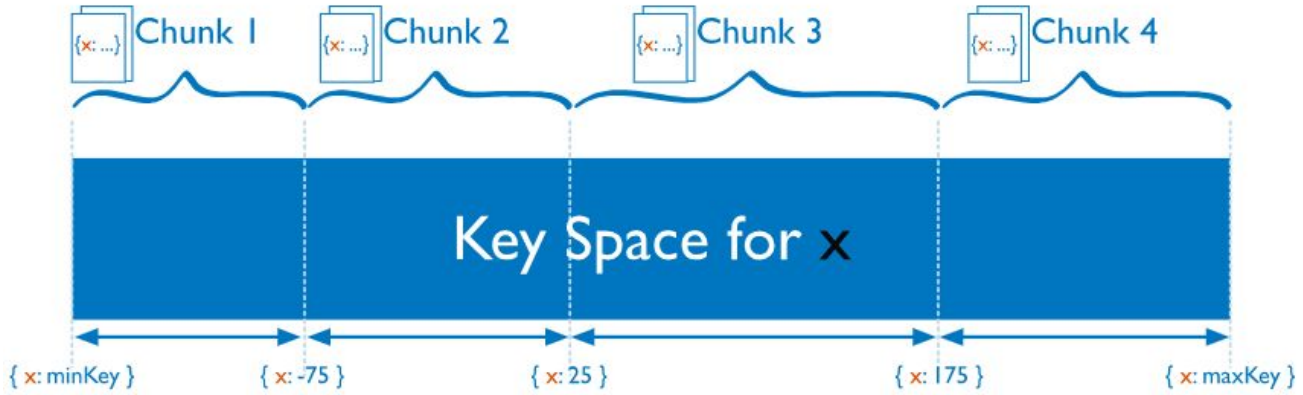
Shard key

- The shard key can directly have an impact on the performance of the cluster. Hence can lead to bottlenecks in applications associated with the cluster.

- To mitigate this, before sharding the collection, the shard key must be created based on:
  - The schema of the data set
  - How the data set is queried

Chunks: Chunks are subsets of shared data. MongoDB separates sharded data into chunks that are distributed across the shards in the shared cluster. Each chunk has an inclusive lower and exclusive upper range based on the shard key. A **balancer** specific for each cluster handles the chunk distribution.

- The balancer runs as a background job and distributes the chunks as needed to achieve an even balance of chunks across all shards. This process is called **even chuck distribution.**




```
sh.splitAt("myDatabase.myCollection", { userId: 100 })
sh.splitAt("myDatabase.myCollection", { userId: 200 })
sh.moveChunk("myDatabase.myCollection", { userId: 150 }, "shard0001")
sh.shardCollection("myDatabase.myCollection", { userId: "hashed" })
```

Choose a Shard Key

- The choice of shard key affects the creation and distribution of chunks across the available shards. The distribution of data affects the efficiency and performance of operations within the sharded cluster.
- The ideal shard key allows MongoDB to distribute documents evenly throughout the cluster while also facilitating common query patterns.
- When you choose your shard key, consider:

- **The cardinality of the shard key:** The cardinality of a shard key determines the maximum number of chunks the balancer can create. Where possible, choose a shard key with high cardinality. A shard key with low cardinality reduces the effectiveness of horizontal scaling in the cluster. A unique shard key value can exist on a single chunk.
- **The frequency with which shard key values occur:** The frequency of the shard key represents how often a given shard key value occurs in the data. If the majority of documents contain only a subset of the possible shard key values, then the chunks storing the documents with those values can become a bottleneck within the cluster.
- **Sharding Query Patterns:** consider your most common query patterns and whether a given shard key covers them.
- You should not choose a key which frequently changes.

Cardinality of the shard key

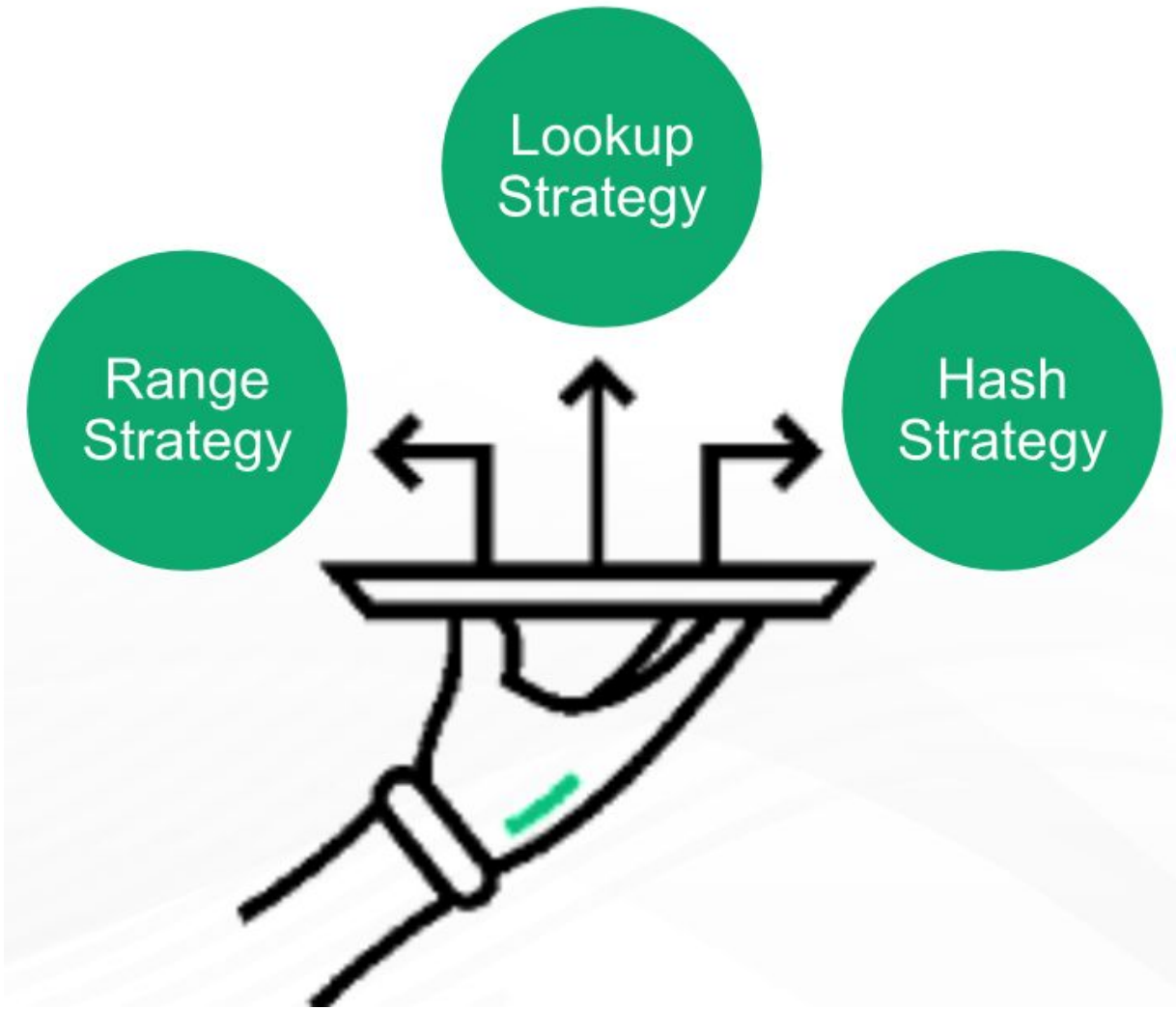
- **Cardinality** refers to the number of unique values in a potential shard key i.e. the number of distinct values for a field used to distribute data across shards.
- It is a critical factor in determining how effectively a database can scale horizontally.
- **High Cardinality:** A shard key with many unique values *(e.g., user_id, email)* allows for fine-grained data distribution and supports a large number of shards.
- **Low Cardinality:** A key with very few unique *values (e.g., is_premium boolean or continent with only 7 values)* limits the total number of shards you can effectively have. For instance, a boolean key limits you to just two shards.
- **Select High-Cardinality Keys:** Choose unique identifiers like user_id or order_id to ensure data can be split into many small, manageable pieces.
- **Compound Shard Keys:** If your primary field has low cardinality, combine it with another field (creating a "compound shard key") to increase total uniqueness.
- **Avoid Monotonic Keys:** Even with high cardinality, avoid keys like timestamps that increase sequentially, as all new writes will hit the same shard, creating a bottleneck.

Frequency of the shard key

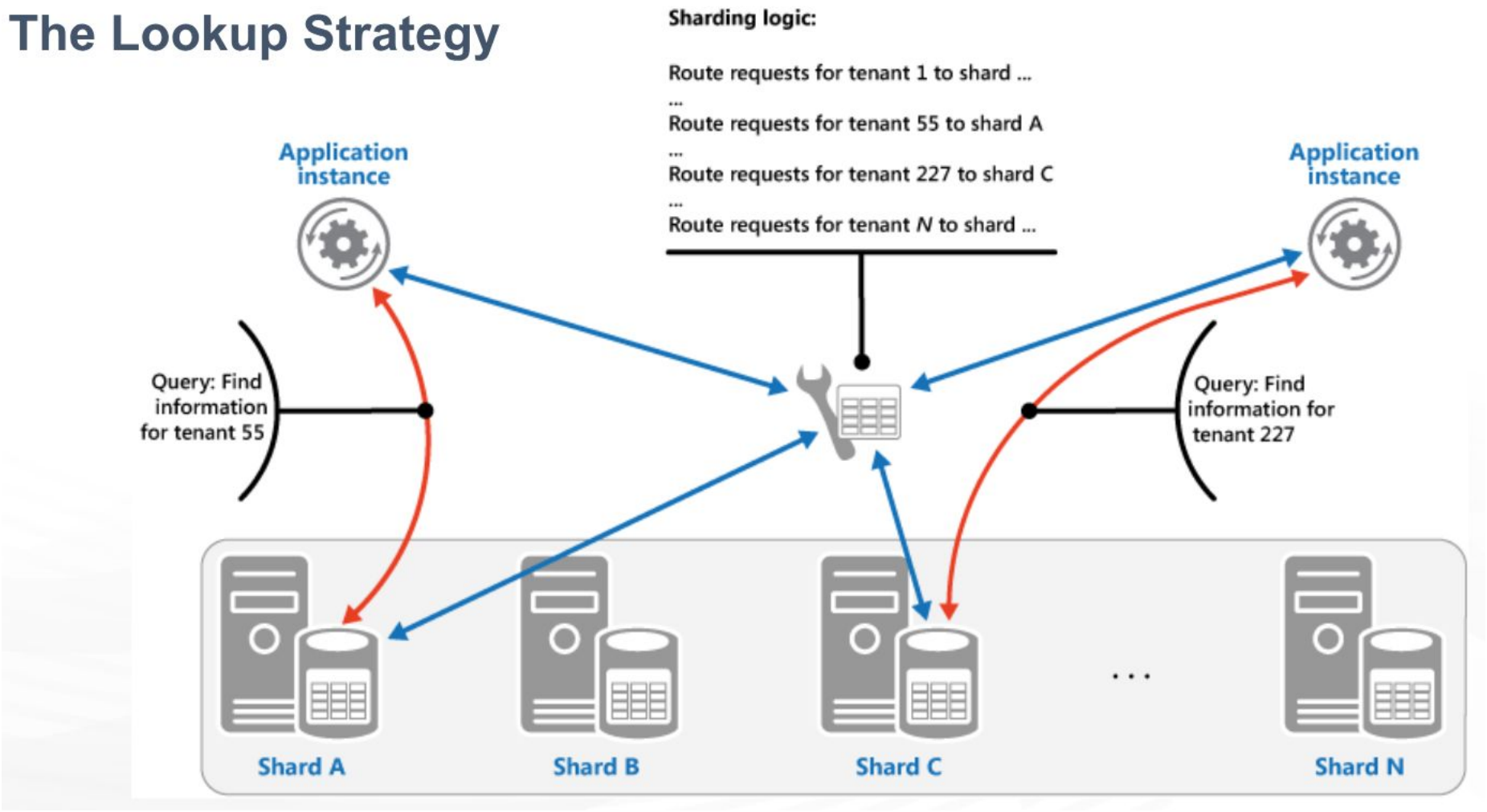
- **Frequency** refers to how often a specific shard key value occurs within the dataset.
- High frequency of a particular key means that a large portion of data shares that same key value, which can lead to data imbalance and performance bottlenecks.
- **Data Distribution:** Frequency measures the distribution of data across possible values. A good shard key should have low frequency—meaning no single value dominates the dataset—to ensure that no single shard becomes overwhelmed.
- **Database Hotspots:** If a shard key has high frequency (e.g., sharding a global application by "Country" where 90% of users are in one country), the shard holding that high-frequency key becomes a "hotspot". This shard will handle most of the reads/writes, causing performance degradation.
- **Impact on Scalability:** High-frequency keys, even if they have high cardinality (many unique values), can result in unevenly sized shards. For instance, in a fitness app, a shard for "ages 30–45" might have much higher frequency (more data) than "ages 80+".
- **Frequency vs. Cardinality**
  - **Cardinality:** The *number* of unique values a shard key has (e.g., user_id has high cardinality; gender has low cardinality).
  - **Frequency:** *How often* each of those unique values appears in the dataset
- **Ideal Scenario:** High Cardinality + Low Frequency.

Sharding strategies

- There are three major strategies commonly used when selecting the shard key and distributing the data across shards.



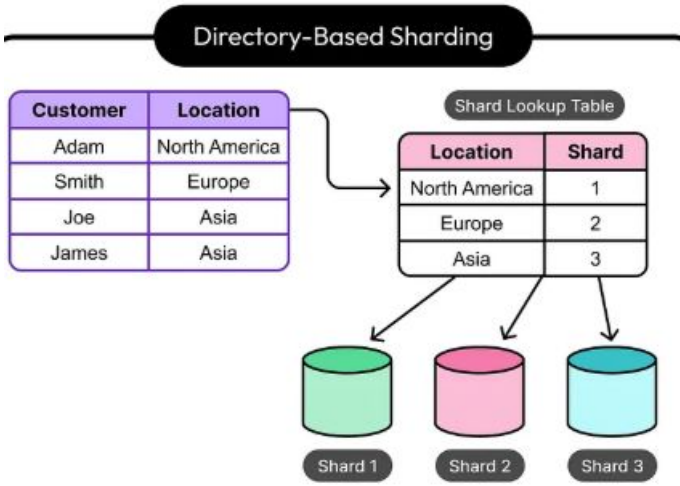
The Lookup Strategy



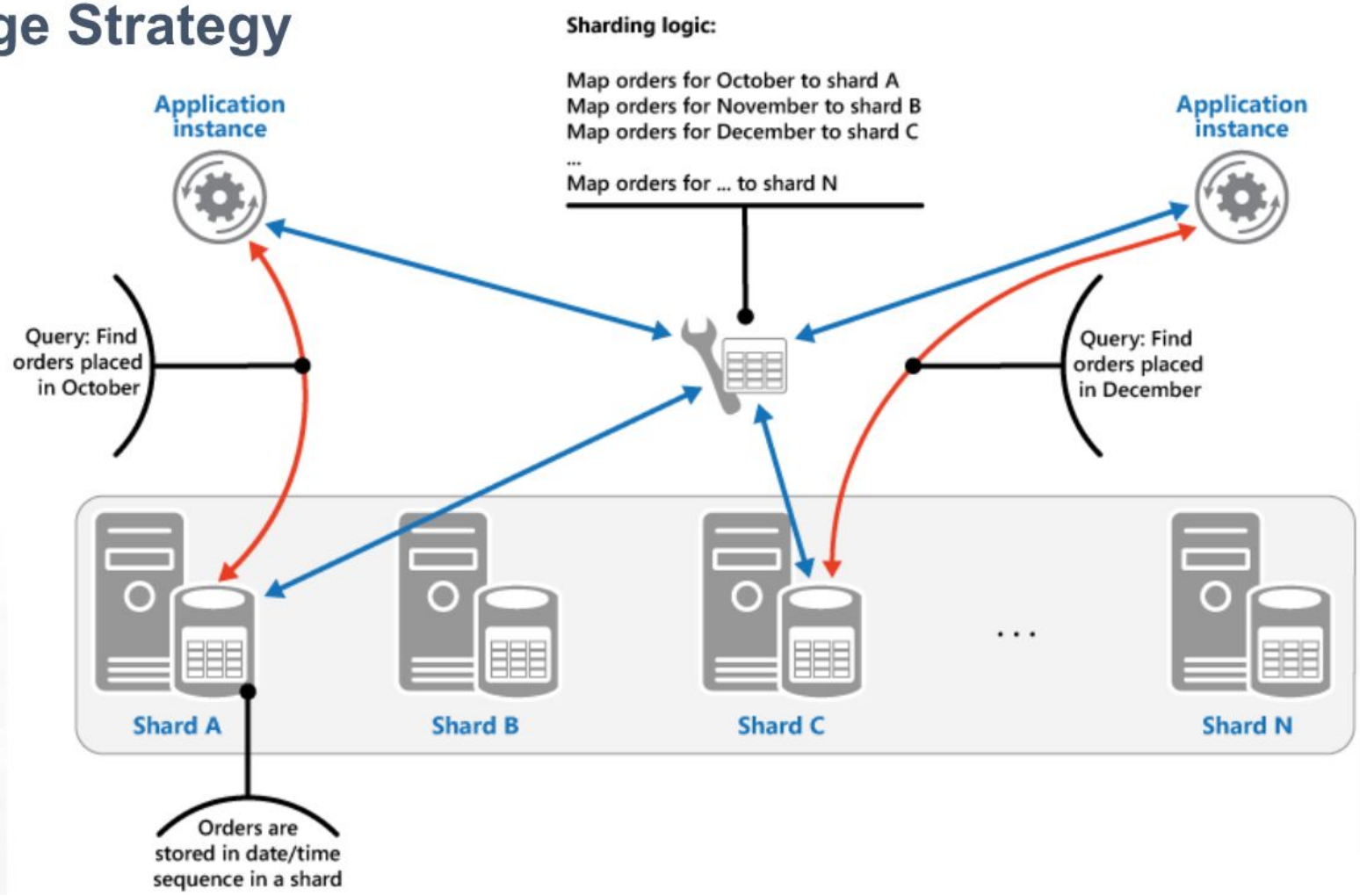
In lookup strategy, the sharding logic implements a map routes a request using the shard key, to retrieve the data that is located in a shard. In a multi-tenant application, the tenant ID is used as the shard key, to store all the data related to a tenant in a shard. This means data for a single tenant will be present in only one shard, and spread across multiple shards. However, multiple tenants can share the same shard. Shard key and physical storage are mapped based on physical shards, where each shard key maps to a physical location.

Look-up Stratergy

- **Lookup-based** sharding (or **directory-based** sharding) uses a centralized lookup table or service to map specific data keys (like user IDs) directly to a specific physical shard.
- This strategy offers maximum flexibility, allowing for custom data placement and easy rebalancing by updating the map, but it requires an extra lookup step for every request.
- Maintains a mapping (lookup table) where Key -> Shard ID.
- **Advantages:** Highly flexible; allows for custom distribution (e.g., placing all European customers on one shard) and easy rebalancing.
- **Disadvantages:** Introduces a single point of failure and bottleneck (the lookup table). Increased latency for queries due to the lookup step.
- **Use Cases:** Ideal for scenarios requiring flexible, non-algorithmic data placement, such as mapping users to specific geographic shards (geosharding).



The Range Strategy



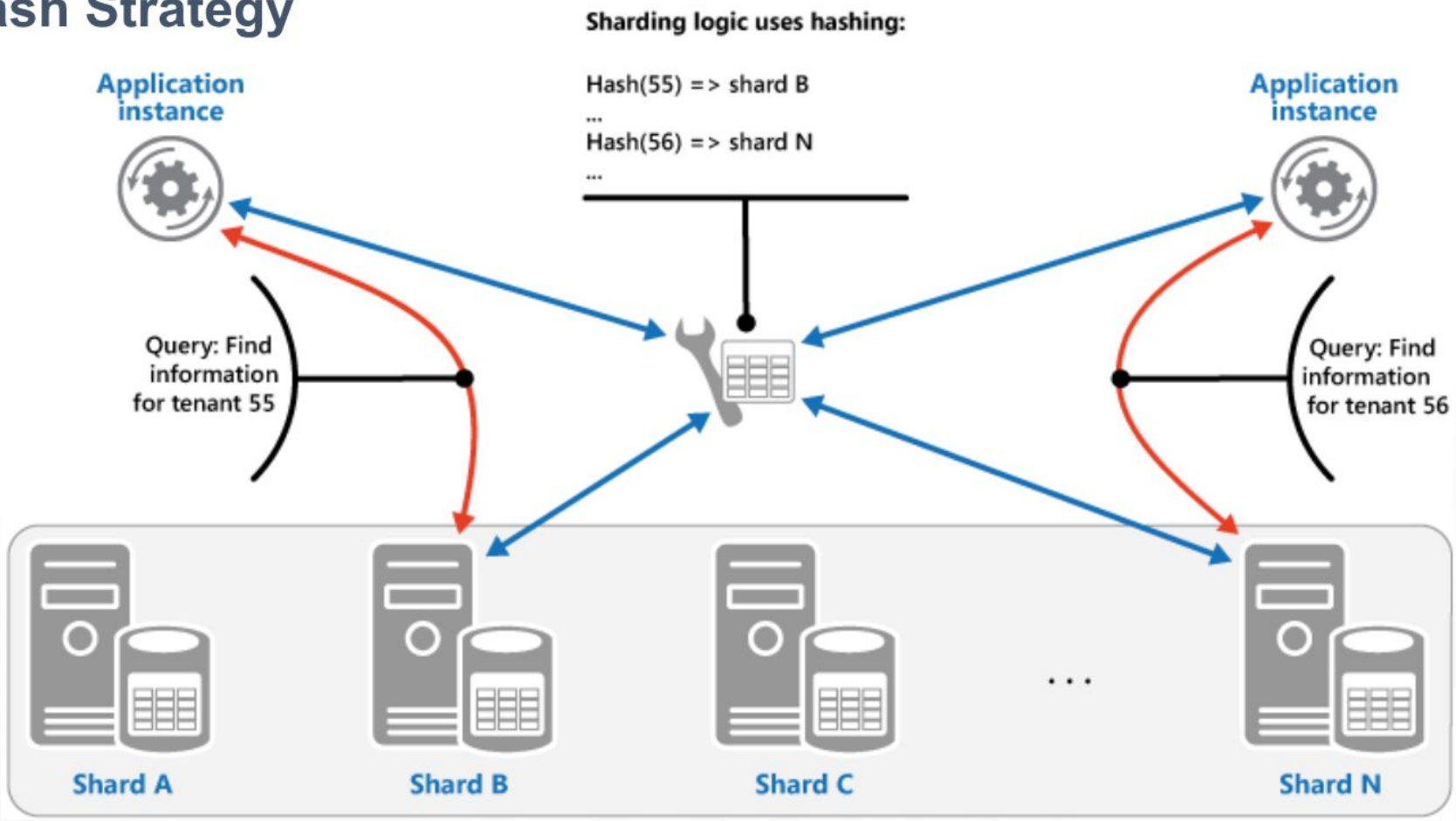
Range strategy groups related items together in the same shard and retrieve them using the shard keys, which are sequential. Range strategy will be an attractive option for applications that frequently retrieve sets of items using range queries (queries that return a set of data items for a shard key that falls within a given range).

- Ranged sharding, or dynamic sharding, takes a field on the record as an input and, based on a predefined range, allocates that record to the appropriate shard. Ranged sharding requires there to be a lookup table or service available for all queries or writes. For example, consider a set of data with IDs that range from 0-50. A simple lookup table might look like the following:

Range	Shard ID
[0, 20)	A
[20, 40)	B
[40, 50]	C

- **The field on which the range is based is also known as the shard key.** Naturally, the choice of shard key, as well as the ranges, are critical in making range-based sharding effective. A poor choice of shard key will lead to unbalanced shards, which leads to decreased performance. An effective shard key will allow for queries to be targeted to a minimum number of shards. In our example above, if we query for all records with IDs 10-30, then only shards A and B will need to be queried.

The Hash Strategy



Hash strategy is helpful in reducing the chance of hotspots (shards that receive a disproportionate amount of load). Data is distributed across the shards in a way that there is a balance between the size of each shard and the average load that each shard will encounter.

The Sharding logic directs the Shard to store an item based on a hash of one or more attributes of the data. The chosen hashing function should distribute data evenly across the shards. This can possibly be achieved by introducing some random element into the computation.

- Algorithmic sharding or hashed sharding, takes a record as an input and applies a hash function or algorithm to it which generates an output or hash value. This output is then used to allocate each record to the appropriate shard.
- The function can take any subset of values on the record as inputs. Perhaps the simplest example of a hash function is to use the modulus operator with the number of shards, as follows:

  Hash Value=ID % Number of Shards
- This is similar to range-based sharding — a set of fields determines the allocation of the record to a given shard. Hashing the inputs allows more even distribution across shards even when there is not a suitable shard key, and no lookup table needs to be maintained. However, there are a few drawbacks.
- First, query operations for multiple records are more likely to get distributed across multiple shards. Whereas ranged sharding reflects the natural structure of the data across shards, hashed sharding typically disregards the meaning of the data. This is reflected in increased broadcast operation occurrence.
- Second, resharding can be expensive. Any update to the number of shards likely requires rebalancing all shards to moving around records. It will be difficult to do this while avoiding a system outage.

Hashed Sharding

- **Even Data Distribution:** Because input keys are hashed, data is evenly spread, preventing any single shard from becoming a performance bottleneck.
- **Preventing Hotspots:** It is highly effective for write-heavy applications where sequential data (like timestamps) would otherwise overwhelm a single server.
- **Fixed Shard Count Constraint:** A major disadvantage is that adding or removing servers often requires significant data redistribution (re-sharding) because the hash(key) % n formula changes.
- **Reduced Range Queries:** Since related data is scattered across shards, range queries (e.g., "get all users registered today") are inefficient, unlike in range-based sharding.
- **Use Cases:** Best for systems needing high write throughput (e.g., IoT, logging) where data is accessed by a specific ID (e.g., user_id).
- **Hash Sharding Process:**
  - **Select Shard Key:** Choose a high-cardinality field (e.g., user_id or uuid).
  - **Apply Hash Function:** The system computes hash(key).
  - **Determine Shard ID:** The hash result is mapped to a specific shard, often using modulo (e.g., hash(key) % n).

Advantages of Sharding

- **Reads / Writes:** MongoDB distributes the read and write workload across the shards in the sharded cluster, allowing each shard to process a subset of cluster operations.
- Both read and write workloads can be scaled horizontally across the cluster by adding more shards.

- **Storage Capacity:** Sharding distributes data across the shards in the cluster, allowing each shard to contain a subset of the total cluster data.
- As the data set grows, additional shards increase the storage capacity of the cluster.

- **High Availability:** If one or more shard replica sets become completely unavailable, the sharded cluster can continue to perform partial reads and writes.
- That is, while data on the unavailable shard(s) cannot be accessed, reads or writes directed at the available shards can still succeed.

Disadvantages of sharding

- **Query overhead**: Each sharded database must have a separate machine or service which understands how to route a querying operation to the appropriate shard. This introduces additional latency on every operation.

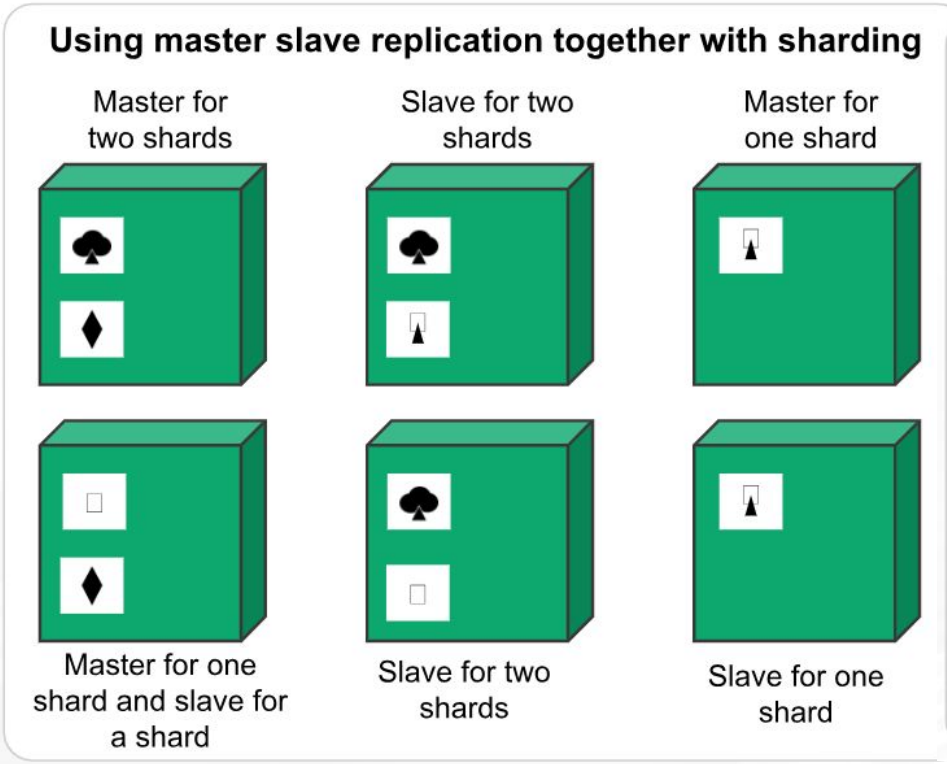
- Furthermore, if the data required for the query is horizontally partitioned across multiple shards, the router must then query each shard and merge the result together. This can make an otherwise simple operation quite expensive and slow down response times.

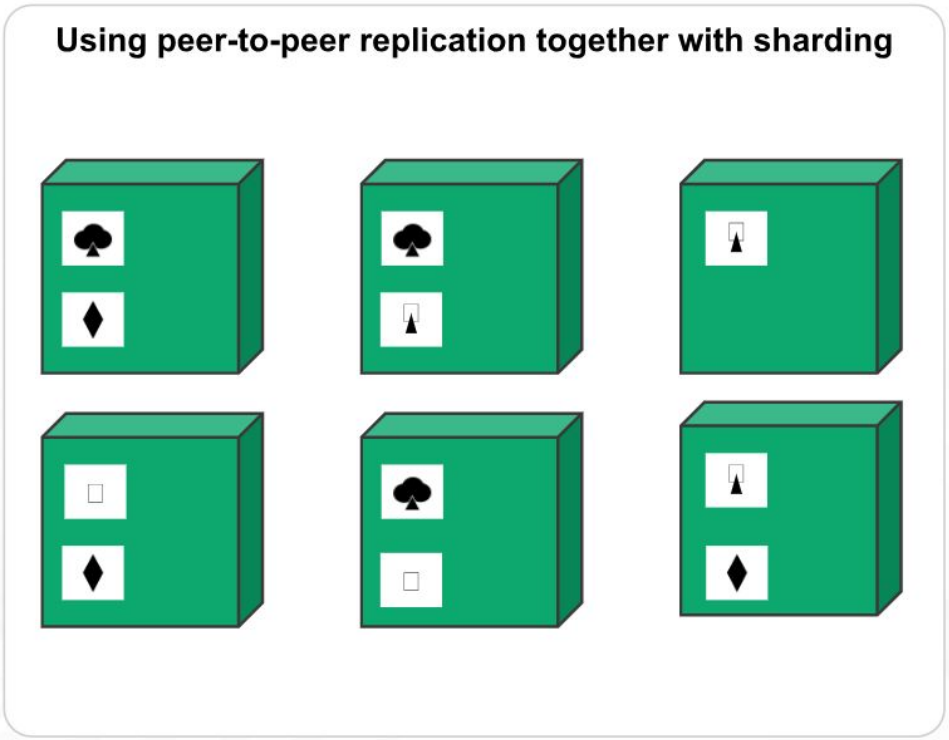
- **Complexity of administration**: With a single unsharded database, only the database server itself requires upkeep and maintenance. With every sharded database, on top of managing the shards themselves, there are additional service nodes to maintain.

- **Increased infrastructure costs**: Sharding by its nature requires additional machines and compute power over a single database server.

- While this allows your database to grow beyond the limits of a single machine, each additional shard comes with higher costs.
- The cost of a distributed database system, especially if it is missing the proper optimization, can be significant.

Combining Replication and Sharding (Sharded Replica Set)





- With a combination of Master-slave replication and sharding, we can have multiple masters, but each data item can have only one master. Based on the configuration, we can choose a node to be the master for some data and slaves for others.

- Using peer-to-peer replication and sharding, there will be tens or hundreds of nodes in a cluster with data shared over them.

Master-slave Replication with Sharding

- **Master (Primary) for a Shard:** The node within a specific shard responsible for all **write operations** (and usually reads) for its subset of data.
- **Slave (Secondary) for a Shard:** One or more nodes that sync with the master to maintain a copy of the shard's data, used for **read scaling** and **failover.**

- **Architecture Example**
  - **Whole Database** = Shard 1 + Shard 2 + Shard 3
  - **Shard 1** = {Master A, Slave A1, Slave A2}
  - **Shard 2** = {Master B, Slave B1, Slave B2}
  - **Shard 3** = {Master C, Slave C1, Slave C2}

  If "Master A" goes down, "Slave A1" or "A2" is elected as the new primary for Shard 1, allowing the application to continue functioning

- **Advantages**
  - **Horizontal Scaling:** Increases write throughput by adding more shards.
  - **Read Throughput:** Increased by adding more slave nodes per shard.
  - **Fault Tolerance:** Data is not lost if a server fails, as replicas (slaves) exist.

Peer-to-Peer Replication with Sharding

- The database is divided into shards (sharding). Each shard is not just one node, but a set of replicas working in a P2P fashion (multi-master).

- **Data Distribution:** Data is partitioned across different sets of servers, so each server acts as a node for a subset of the total data.

- **Consistency:** P2P offers high availability but may result in "eventual consistency." If two nodes receive conflicting writes, complex conflict resolution is required.